

Simple Smart Device Control System

1. Understanding of the Problem Statement

Objective:

The objective of this assignment is to design a simple smart device control system using Python and object-oriented programming (OOP).

Key Requirements:

- The system should handle different devices, starting with Motor and Light, with the ability to add more devices in the future.
- Each device must have Start and Stop operations and maintain an internal ON/OFF state.
- The internal state must be protected, so external code cannot modify it directly.
- Each device's internal state (`_is_on`) is initially False.
- There should be a Controller class that can accept any device and operate it without depending on the specific device type.
- External code should be able to check the device status using a safe method (`is_on()`).

Constraints / Key Points:

- Devices should be independent (work on their own) and modular (can be added, removed, or updated without affecting the rest of the system).
- The system must be scalable, so new devices can be added without changing existing controller logic.
- The design should follow OOP principles such as encapsulation, inheritance, polymorphism, and abstraction.

2. Approach and Design

Device Base Class (Abstraction):

- A base class `Device` defines the interface for all devices.
- Methods `start()` and `stop()` are abstract (implemented with `raise NotImplementedError`) so that every subclass must provide its own implementation.
- The internal state `_is_on` is protected and can only be accessed externally via the `is_on()` method.

Device Subclasses (Inheritance & Polymorphism):

- `Motor` and `Light` classes inherit from `Device`.
- Each subclass implements `start()` and `stop()` differently to show unique behavior.
- This demonstrates polymorphism, as the same controller code can operate any device using the same method calls.

Controller Class:

- The `Controller` class is independent of specific device types.
- It accepts a device object and calls its `start()` and `stop()` methods.
- This ensures extensibility, as new devices can be added without modifying the controller.

Design Decisions:

- **Encapsulation:** `_is_on` state is protected; external code cannot change it directly.
- **Abstraction:** Base `Device` class enforces a standard interface for all devices.

- **Polymorphism:** Subclasses implement `start()` and `stop()` differently, but controller code remains the same.
- **Scalability:** Adding new devices only requires creating a subclass with `start()` and `stop()`.

Justification:

- This OOP-based design was chosen over a procedural approach because it allows encapsulation, polymorphism, and easy extensibility without modifying existing code.
- Procedural design would require separate functions for each device, which becomes hard to maintain as devices are added.

3. Solution Explanation

Step-by-step Implementation:

Base Device Class:

- Contains `_is_on` internal state, initially set to `False`.
- `start()` and `stop()` are abstract methods that must be implemented in subclasses.
- `is_on()` method allows external code to check the device's current state safely.

Device Subclasses:

- **Motor:**
 - `start()` → sets `_is_on = True`, prints "Motor has started"
 - `stop()` → sets `_is_on = False`, prints "Motor has stopped"
- **Light:**
 - `start()` → sets `_is_on = True`, prints "Light switched on"

- `stop()` → sets `_is_on = False`, prints "Light switched off"

Controller Class:

- Accepts any device object.
- Calls `start()` and `stop()` on the device.
- Works correctly for current and future device types.

Driver Code (`main.py`):

- Create `Motor` and `Light` objects.
- Create a `Controller` for each device.
- Run the controller for each device.

Expected Output:

```
Motor has started
Motor has stopped
Light switched on
Light switched off
```

Checking Device Status:

- External code can call `device.is_on()` to verify the device's internal state.
- Example:

```
motor = Motor()

print(motor.is_on())  # False
```

Testing & Verification:

- The solution was tested by running `main.py` and confirming the output matches the expected results.
- The `is_on()` method correctly reflects the internal state of each device.
- No execution errors were observed.